

```

/*
FFT, CW Decode for uBITX
KD8CEC, Ian Lee
WC8C, Dennis Cabell
Version : 2
-----
License : See fftfunctions.cpp for FFT and CW Decode.
*****/

#include <Wire.h>
#include <EEPROM.h>
#include "i2cmeter2.h"
// #include "AltSoftSerial.h"
#include "CWDecoder.h"
#include "fftfunctions.h"
#include "NextionProtocol.h"
#include "PowerSwr.h"
#include "Configuration.h"

#ifdef USE_SW_SERIAL
    AltSoftSerial SERIAL_OUTPUT;
    NextionProtocol nextion = NextionProtocol(SERIAL_OUTPUT);
#else
    NextionProtocol nextion;
#endif

#ifdef ENABLE_POWER_SWR_METER
    PowerSwr powerSwrCalculator;
#endif

CWDecoder decoder;

int FFTReal[SAMPLESIZE];
int ADC_DIFF = 0;
volatile byte I2CCommand = 0; // volatile because I2C interrupt changes this

int GrepADC(int readSampleCount, int *readArray)
{
    #ifdef USE_SW_SERIAL
        // if there is data to still output then return so normal forwarding can continue.
        // only want to sample if the SW Serial output is empty so extra interrupts dont
        // disrupt sample timing.
        if (!SERIAL_OUTPUT.TxBufferIsEmpty())
        {
            return -1;
        }
    #endif
}

```

```

unsigned long currentms = 0;
unsigned long newCurrentms = 0;
int ADC_MAX = 0;
int ADC_MIN = 30000;

analogRead(SIGNAL_METER_ADC); // initial unused value helps accuracy since ADC may
have been switched to read power & swr.
newCurrentms = micros();

for(int i=0; i < readSampleCount; i++)
{
    if (nextion.SerialDataToProcess())
    {
        // if something has come in then terminate the FFT sample so incoming data can
        be processed.
        // this is so that frequency tuning does no lag when in FFT mode.
        // returning -1 instead of ADC_DIFF indicates that the sample is incomplete.
        return -1;
    }

    currentms = newCurrentms;
    readArray[i] = analogRead(SIGNAL_METER_ADC);
    ADC_MAX = max(ADC_MAX, readArray[i]);
    ADC_MIN = min(ADC_MIN, readArray[i]);

    while((newCurrentms - currentms) < SAMPLE_INTERVAL){newCurrentms = micros();}
} //end of for

return ((ADC_MAX - ADC_MIN) * SMETER_RESOLUTION_MULTIPLIER) / SMETER_GAIN; //
return ADC_DIFF
}

void setup()
{
    Wire.begin(I2CMETER_ADDR); //j : S-Meter Slave Address
    Wire.onReceive(I2CReceiveEvent);
    Wire.onRequest(I2CRequestEvent);

    Serial.begin(SERIAL_SPEED, SERIAL_8N1);
    Serial.flush();

#ifdef USE_SW_SERIAL
    SERIAL_OUTPUT.begin(SERIAL_SPEED);
#endif

    nextion.Initialize();

```

```

    //spend time just forwarding data while everything starts up. There is a lot of
    initial data to transfer.
    unsigned long startMillis = millis();
    while (millis() - startMillis < 2000)
    {
        nextion.ForwardData();
    }
}

void loop()
{
    while (nextion.SerialDataToProcess())
    {
        // This will continue to loop until and entire command is forwarded.
        // Only add things here if it must absolutely interleave with forwarding a
        command from the uBitx.
        // Note that the I2C Request work cannot be moved here because Wire.Write must
        stay in the ISR to function correctly.
        nextion.ForwardData(); // this will not return until the input UART is empty
    }

    //=====
    //Processing for when a config response has needs to be sent
    //=====
    if (nextion.ResponseConfig())
    {
        return; // if anything was sent to nextion, then start loop over so any data that
        may have come in can be processed.
    }

    //=====
    //Processing for when the frequency has changed
    //=====
    if (nextion.frequencyHasChanged)
    {
        // TBD: Call process to do when the frequency has changed (e.x. output to an
        antenna tuner)

        nextion.frequencyHasChanged = false;
        return; //Start a new loop so more serial data can be processed
    }

    //=====
    //TRANSCIEVER STATUS : TX
    //=====
    #ifdef ENABLE_POWER_SWR_METER
    static uint8_t pwrSwrResetCounter = 0; // when TX disabled, how many times to try to
    reset the power and SWR to ensure Nextion gets it

```

```

#endifif
if (nextion.TXStatus == 1 || !nextion.nextionIsConnected) // TX Mode, or always
read if there is no nextion so values are available if requested via I2C
{
    #ifdef ENABLE_POWER_SWR_METER
        powerSwrCalculator.PowerSwrCalculation(nextion.L_vfoCurr);
        if (nextion.nextionIsConnected)
        {
            nextion.SendPowerSwr(powerSwrCalculator.power, powerSwrCalculator.swr, true);
            pwrSwrResetCounter = POWER_SWR_RESET_RETRY_COUNT;
            return; //Do not process ADC, FFT, Decode Morse or RTTY, only Power and SWR
Data Return
        }
    #endif
}
else
{
    #ifdef ENABLE_POWER_SWR_METER
        if (nextion.nextionIsConnected && pwrSwrResetCounter > 0)
        {
            // reset the power and SWR meter when state changes out of TX mode
            if (!nextion.SendPowerSwr(0.0, 1.0, false))
            {
                return; // if incoming data aborted the send, return so it can be
processed
            }
            pwrSwrResetCounter--;
        }
    #endif
}

//=====
//TRANSCIEVER STATUS : RX or no Nextion
//=====
//=====
// ADC Sampling
//=====
int new_ADC_DIFF = GrepADC((nextion.DSPTYPE == 2 || nextion.DSPTYPE == 3) ?
DECODE_MORSE_SAMPLESIZE : SAMPLESIZE, FFTReal);
if (new_ADC_DIFF < 0)
{
    return; // If the sample was not completed, then return so incoming data can be
processed in the next loop.
}

noInterrupts(); // disable interrupts while saving these so that the I2C ISR
cant read them while they are changing.
ADC_DIFF = new_ADC_DIFF;

```

```

interrupts();
nextion.CalculateScaledSMeter(ADC_DIFF);

//=====================================================
// DSP Routine - only if there is a Nextion

//=====================================================
if (nextion.nextionIsConnected)
{
    if (nextion.DSPTYPE == 0)
    {
        nextion.SendScaledSMeter(FastMeterToUartInterval);
    }
    else if (nextion.DSPTYPE == 1 && nextion.LastSendWaitTimeElapsed()) //Spectrum :
FFT => Send To UART
    {
        nextion.SendScaledSMeter(FFTMeterToUartInterval);
        nextion.SendFFTData(SAMPLESIZE, FFTReal); // if new data comes in, the send gets
aborted and control returns so the loop can be started over.
    }
    else if (nextion.DSPTYPE == 2)          //Decode Morse
    {
        decoder.Decode_Morse(GetMagnatude(nextion.coeff, DECODE_MORSE_SAMPLESIZE,
FFTReal), nextion.magnitudelimit_low);
        if (!nextion.SerialDataToProcess()) // only send decoded data if nothing new
has come in to process.
        {
            if (nextion.SendDecodeCharacters(decoder.GetDecodedCharacters()))
            {
                decoder.ClearDecodedCharacters(); // if send succeeded clear the characters
            }
            nextion.SendScaledSMeter(DecodeMeterToUartInterval); // Note that if characters
were sent then this will defer until LAST_TIME_INTERVAL has elapsed. They wont send
back to back
        }
    }
}
} //end of main loop

void I2CReceiveEvent(int)
// void I2CReceiveEvent(void)
{
    int readCommand = 0; // byte를 읽어 int로 변환

    while(Wire.available() > 0)    // for Last command
    {

```

```

    readCommand = Wire.read();
}

if (0x50 <= readCommand && readCommand <= 0x59)
{
    I2CCommand = readCommand;
}
}

void I2CRequestEvent(void)
{
    unsigned long startMicros = micros();
    byte command = I2CCommand; // make a copy in case it case the other interrupt
    changes it while this is executing

    if (command == I2CMETER_CALCS)
    {
        Wire.write(nextion.scaledSMeter/SMETER_RESOLUTION_MULTIPLIER); // only sends low
        resolution SMeter
    }
    else if (command == I2CMETER_UNCALCS)
    {
        int maxValue = 0;
        int minValue = 30000;
        int adcValue = 0;

        while((micros() - startMicros) < 900) // read as many as we can in 1 allotted
        time. Longer can cause issues with the timers
        {
            adcValue = analogRead(SIGNAL_METER_ADC);
            maxValue = max(adcValue, maxValue);
            minValue = min(adcValue, minValue);
        }

        // cast to a long because this theoretically can be 1023 * 1023, then cast it back
        down after the constrain
        // Not sure why it is being squared - maintaining behavior of original code.
        long sampleDifference = (float)(maxValue - minValue) / SMETER_GAIN;
        Wire.write((uint8_t)constrain(sampleDifference * sampleDifference, 0, 255));
    }
    /* Notes on power and SWR
    *
    * Original code sends an int, so this does too, but this will get converted to 8
    bits if the value is > 255
    * constraining to 255 seems too small. Ask Dr. Lee
    *
    * Dont need to read if in RX mode, just send zero because when not transmitting the
    value should always read zero
    */
}

```

```

    * however if there is no nextion connected (i.e. there is no serial input), then
the TX status is unknown
    * so the data needs sent.
    *
    * Note that the analog reads happen in the main loop and the values are saved so
they are available here.
    * This is to keep the interrupt fast because the analogRead is a bit slow.
    */
else if (command == I2CMETER_CALCP)    // POWER
{
    int value = 0;
    #ifdef ENABLE_POWER_SWR_METER
        if (nextion.TXStatus == 1 || !nextion.nextionIsConnected) //TX Mode or there
isn't a nextion
        {
            value = powerSwrCalculator.powerIn;
        }
    #endif
    Wire.write(value); //POWER
}
else if (command == I2CMETER_CALCR)    // REV POWER
{
    int value = 0;
    #ifdef ENABLE_POWER_SWR_METER
        if (nextion.TXStatus == 1 || !nextion.nextionIsConnected) //TX Mode or there
isn't a nextion
        {
            value = powerSwrCalculator.revPowerIn;
        }
    #endif
    Wire.write(value); //Reverse Power
}
}

```